

MySQL & Python Ride Share App

CPSC408

Suppose that a new start-up is trying to create a rideshare app and hires you to design their database. Think about the main information that the app must keep track of and create some deliverables to show your client as you go.

You must create:

1. An ER diagram
2. A database schema with all tables and attributes
3. A database on MySQL locally
4. Sample data within your database to test
5. A simple interactive python program.

For the interactive part:

1. You will ask the user if they are a new user, a rider, or a driver.
2. If the user is a new user, let them create either a rider or driver account (see below for requirements on each account)
3. If the user is not new, have them log into a specific rider or driver account using an ID.
4. If the user is a driver, you will give them the following options:
 - a. View Rating: This will show the driver their current rating. This will be the average rating of all rides they have given.
 - b. View Rides: This will show the driver the list of all rides they have given.
 - c. Activate/Deactivate Driver Mode: This updates a flag on their record that they are able to drive and are now accepting ride requests
5. If the user is a rider, you will give them the following options:
 - a. View Rides: This will show the rider the list of all rides they have taken
 - b. Find a driver: Match the rider with a driver that has their driver mode activated
 - i. The rider will then provide the following info:
 1. Pick up location
 2. Drop off location
 - ii. Then, you will create a ride and provide the rider with a ride ID
 - iii. You will then send the rider back to their options menu
 - c. Rate my driver:
 - i. You will look up the rider's most recent ride and get the driver's ID
 - ii. You will then print this information to the user and ask if it is the correct ride
 - iii. If it is not the correct ride, you will have them enter the rideID of the ride they want to rate. Print that ride's information and have them confirm.
 - iv. Store the rating the rider gave on the ride record.

Any database schema you think can accomplish the above use case that is correctly normalized and planned is allowed, as long as you can justify your design decisions to me. Feel free to add any more attributes or functionality on top of this as you see fit to make a good application to show your theoretical client!

Ensure you plan using the correct steps:

1. Create your ER Diagram
2. Assign relationships and then determine their cardinality
3. Use our slides on referential integrity and foreign keys to ensure you translate those relationships correctly into a schema
4. Make sure you check that your database schema is normalized to the 3rd normal form by identifying all functional dependencies, primary keys, and foreign keys.
5. Go on to create your schema and database + the app around it similar to how we built the playlist app.

You may come to me at any time asking to confirm your plan looks good before you implement it (and in fact I enthusiastically recommend it!). If you implement a non-normalized database, or a database where the foreign key placement does not match the relationships on your ER diagram, I **WILL** make you redo it to get full points.